

Day 91: Weekly Revision — Databases and Math Introduction

1. Day Number

Day 91

2. Topic Name

Revision of Days 85–90: Databases and Math Introduction

Today you will revise how data can be stored in different database systems and how the same data can later be represented mathematically for machine learning.

3. Connection

Over the last few days, you learned two connected ideas.

First, applications need ways to **store and retrieve data**:

```
SQL / MySQL
MongoDB
Redis
```

Second, machine learning needs ways to represent numerical data mathematically:

```
variables
equations
vectors
matrices
```

So the overall connection is:

```
Store data
  ↓
Retrieve data
  ↓
Prepare numerical features
  ↓
Represent them as vectors/matrices
  ↓
Use them in ML
```

4. Revision Summary of Days 85–90

Day	Topic	Main Idea
85	SQL CRUD + JOIN	Add, read, update, delete, and combine relational data

Day	Topic	Main Idea
86	MySQL + Python	Python can send SQL queries to a MySQL server
87	MongoDB	Store dictionary-like documents in collections
88	Redis	Store simple key-value data and use caching
89	Basic Algebra	Understand variables, coefficients, constants, and linear relationships
90	Vectors + Matrices	Represent observations and features numerically

Day 85

You learned CRUD:

```
Create → INSERT
Read   → SELECT
Update → UPDATE
Delete → DELETE
```

You also learned that a `JOIN` can combine related tables.

For example:

```
customers
+
orders
```

can be connected using:

```
customer_id
```

Day 86

You learned the basic flow between Python and MySQL:

```
Python
→ connection
→ cursor
→ SQL query
→ MySQL
→ result
→ Python
```

A simple `SELECT` query can return rows that Python processes.

Day 87

You learned that MongoDB stores **documents** inside **collections**.

Example document:

```
{
  "name": "Asha",
  "age": 28,
  "city": "Bengaluru"
}
```

The basic hierarchy is:

```
Database
  ↓
Collection
  ↓
Document
```

Day 88

You learned Redis as a simple key-value store.

For example:

```
customer:1:city → Bengaluru
```

You also learned the cache idea:

```
Check Redis
  ↓
Found? → use cached value
  ↓
Not found
  ↓
Read main database
```

Day 89

You learned simple algebra such as:

```
output = coefficient × input + constant
```

For example:

```
delivery cost
=
price per km × distance
+
base charge
```

These ideas prepare you for Linear Regression.

Day 90

You learned:

```
scalar → one number
vector → several related numbers
matrix → several vectors together
```

For example, one customer might have:

```
[28, 50000]
```

where:

```
28      → age
50000   → annual income
```

Multiple customers can form a matrix.

5. Important Topics

The main topics to remember are:

- SQL CRUD
- `INNER JOIN`
- MySQL
- database connections
- MongoDB documents
- Redis keys and values
- caching
- variables and constants
- coefficients
- vectors
- matrices
- rows, columns, and shape

A useful mental map is:

Databases

- SQL/MySQL → tables
- MongoDB → documents
- Redis → key-value pairs

Math for ML

- Algebra → relationships between values
- Vector → one observation
- Matrix → many observations

6. Foundational Notes

Different tools can represent the **same real-world information** differently.

Suppose we have a customer:

```
Customer ID = 1
Name        = Asha
Age         = 28
City        = Bengaluru
Spending    = 4500
```

The real customer has not changed.

Only the **representation** changes.

A relational database may use a row.

MongoDB may use a document.

Redis may store one frequently needed value using a key.

Machine learning may ignore the name and city for the moment and use only numerical features such as:

```
[28, 4500]
```

This is an important data-science idea:

The same real-world object can have different representations depending on what the application needs.

7. Easy Example

Consider this customer:

```
ID      = 10
Name     = Ravi
Age      = 30
```

```
City      = Chennai
Spending = 6000
```

SQL Representation

A `customers` table might contain:

customer_id	name	age	city	spending
10	Ravi	30	Chennai	6000

MongoDB Representation

The same customer could be:

```
{
  "_id": 10,
  "name": "Ravi",
  "age": 30,
  "city": "Chennai",
  "spending": 6000
}
```

Redis Representation

If the application frequently needs Ravi's spending:

```
customer:10:spending → 6000
```

ML Feature Vector

If an ML model uses only:

```
age
spending
```

then Ravi could become:

```
[30, 6000]
```

Notice how the same customer appears differently depending on the task.

8. Revision Problem Statement

Use this tiny customer:

```
Customer ID = 5
Name        = Meera
Age         = 35
```

City	= Delhi
Spending	= 7500

Conceptually represent this customer in four ways:

SQL: Show how the customer could appear as one row in a `customers` table.

MongoDB: Show how the customer could appear as one document.

Redis: Choose one useful value, such as spending or city, and represent it as a key-value pair.

Machine Learning: Assume the model uses only `age` and `spending`. Represent the customer as one feature vector.

Keep the exercise to just this one customer.

9. Concepts Used

This revision combines:

database
table
row
column
document
field
key
value
cache
feature
vector
observation

The important relationship is:

Real-world customer
↓
different representations
↓
SQL / MongoDB / Redis / ML

10. Thought Process

Start with the meaning of the data, not the technology.

For the customer:

```
ID = 5
Name = Meera
Age = 35
City = Delhi
Spending = 7500
```

First ask:

| If this were relational data, what columns would I need?

That gives the SQL representation.

Then ask:

| If this were a dictionary-like object, what fields and values would I use?

That gives the MongoDB representation.

Then ask:

| What single value might an application need quickly?

That gives a possible Redis cache entry.

Finally ask:

| Which numerical values would an ML model use as features?

If the model uses `age` and `spending`, then your feature vector contains exactly those two values, in that order.

11. Beginner-Friendly Pseudocode / Conceptual Steps

START

take one customer

SQL:

place customer values into columns
create one row

MongoDB:

create field-value pairs
store as one document

Redis:

choose one frequently needed value
create a descriptive key
store the value

Machine Learning:

```
choose numerical features
place them in a fixed order
create one feature vector
```

END

A compact summary is:

```
SQL      → row
MongoDB  → document
Redis    → key → value
ML       → feature vector
```

12. Suggested Solving Approach: Conceptual Comparison

Do not write a large Python program for this revision.

Instead, take one customer and compare how each technology views the information.

For example:

```
Customer
  ↓
SQL: structured row

Customer
  ↓
MongoDB: document

Customer property
  ↓
Redis: cached key-value

Customer numerical features
  ↓
ML: vector
```

This helps you understand the purpose of each representation instead of memorizing syntax.

13. Easy Edge Cases

Missing Field

Suppose the customer's city is unknown.

A SQL row might have a missing or `NULL` city.

A MongoDB document might have a missing field or a value representing missing data.

Before ML, you would eventually need to decide how to handle that missing information.

Customer Without an Order

In SQL, the customer may exist even if there are no matching rows in an `orders` table.

An `INNER JOIN` with orders would not show that customer unless a matching order exists.

Redis Key Missing

If:

```
customer:5:spending
```

is not present in Redis, it may simply be a cache miss.

The application can obtain the value from the main database.

Only One ML Feature

If the model uses only age, the feature vector could contain just:

```
[35]
```

It is still a vector, just with one feature.

14. Common Mistakes to Avoid

- Thinking Redis, MongoDB, and MySQL organize data in exactly the same way.
- Assuming Redis must replace the main database; it is often used alongside one as a cache.
- Forgetting that `UPDATE` and `DELETE` often need careful `WHERE` conditions.
- Joining tables using unrelated columns instead of the shared ID.
- Thinking MongoDB documents are exactly the same as Python dictionaries; they are similar representations, but one is stored in the database.
- Confusing an ML feature vector with the complete customer record.
- Changing the order of features between observations. If one vector is `[age, spending]`, keep that same order for every customer.

15. Quick Self-Check Questions

1. Which SQL command changes an existing row?
2. What does an `INNER JOIN` do at a basic level?
3. In MongoDB, what is the equivalent beginner-level idea to a table and row?
4. If Redis stores:

```
product:10:price → 500
```

which part is the key and which part is the value?

5. If one customer has features:

```
age = 25  
spending = 4000
```

what would the feature vector look like if the feature order is `[age, spending]`?

Try answering them without looking back first.

16. Hint Only

For the revision exercise, start with:

```
Meera  
Age = 35  
City = Delhi  
Spending = 7500
```

Think:

```
SQL  
→ Which values become columns in one row?  
  
MongoDB  
→ How would I write these as field-value pairs?  
  
Redis  
→ Which one value could I give a descriptive key?  
  
ML  
→ Which two numerical values were selected as features?
```

For the ML part, remember that the requested feature order is:

```
[age, spending]
```

The key lesson for **Day 91** is:

Databases help us store and retrieve real-world information, while vectors and matrices give us numerical representations that machine-learning models can work with.